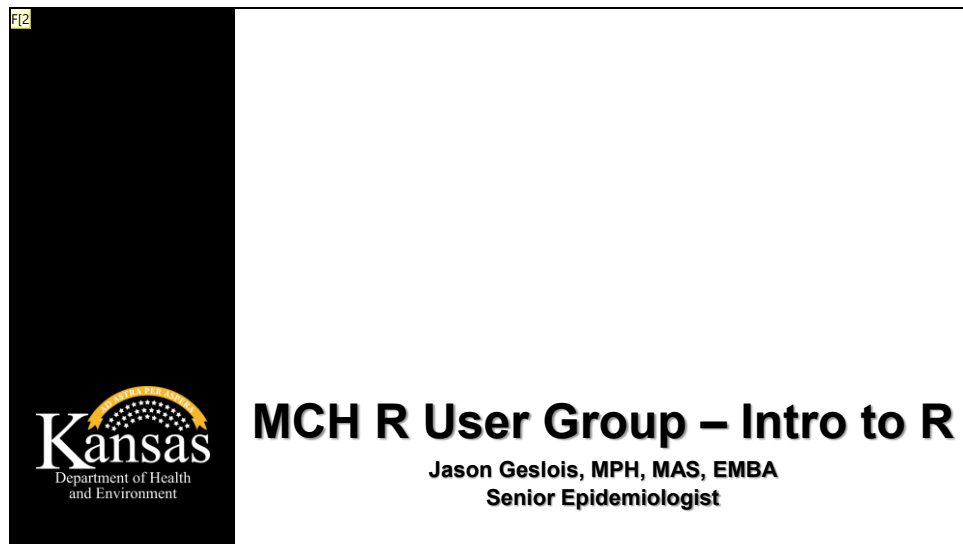


Slide 1



Hello again everyone, I'm Jason Geslois, Senior Epidemiologist at Kansas Department of Health and Environment supporting our Title V and Maternal Child Health program. Today I'm going to walk through an introduction to R, and some of the beginner concepts that you may or may not know coming from SAS. We've briefly talked some of this in other calls, but today, we'll go through the basics, using the interface, and examples of some code. At the end we will do a short demonstration in RStudio to see and learn how to navigate within it. I used NotebookLM AI (<https://notebooklm.google.com/>) to generate some pictures to help better illustrate different concepts for you in this presentation.

(The audience is approximately 6-10 people, 2-3 other state health depts, with epi and data staff focused in the MCH and PRAMS roles.) – April 27, 2026

Slide 2



MCH R User Group – Intro to R

Translating the Philosophy: SAS vs. R

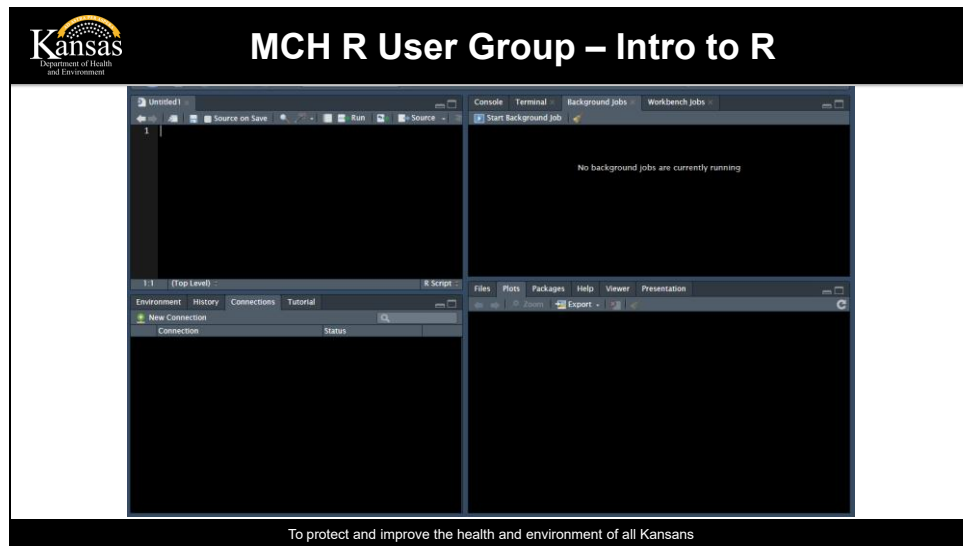
Dimension	Legacy (SAS)	Modern (R)
Data Storage	 Disk-based tables. Persistent storage.	 In-memory Data Frames. Active RAM.
Execution	 Sequential PROCs & DATA steps.	 Interactive, object-oriented functions.
Ecosystem	 Proprietary, built-in modules.	 Open-source, community Packages.
Workspace	 Libraries and Libnames.	 Self-contained RStudio Projects.

#NoNotebookLM

To protect and improve the health and environment of all Kansans

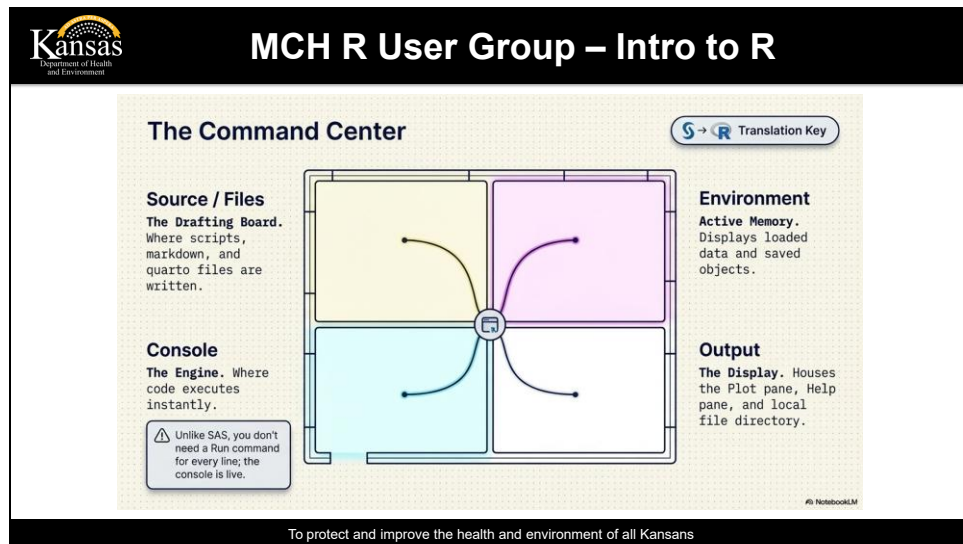
For those coming from a SAS background, there are a few different concepts to understand and learn that work slightly differently in R. First, is that SAS uses hard drive space when working with your data. Meaning it reads and processes all your data on your hard drive. Whereas in R, this is done in memory. For most things you won't notice a difference until you start either needing fast code, or large datasets, and we won't get into either of these examples, but in general, just know the concept of working in the data is done differently for when and how you save your data. In SAS, execution of steps is strict, meaning, if you miss a semicolon, and you are trying to create a dataset, it will fail to create that dataset. R is functional, meaning each object is evaluated immediately and then stored in memory, so the creation of some dataset objects can still exist while others may fail and not need the full code to execute perfectly to create all of them. R is also open source, meaning you can inspect the entire code, including the internal code that R is built from, where SAS is proprietary and built by SAS company. In your workspace R uses Projects compared to SAS' libraries and libnames, and we'll touch more on that in a moment.

Slide 3



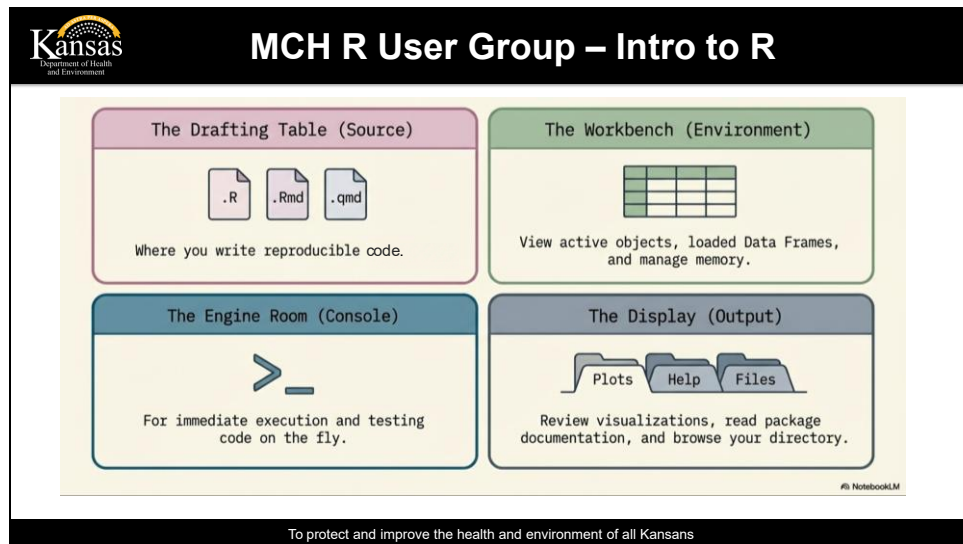
This is the traditional 4 pane view of the RStudio interface that it typically starts with. You can rearrange these panes and colors in the settings, so if yours looks slightly different, that's ok. Each pane, has different tabs, and we will go through a couple of those in our demo.

Slide 4

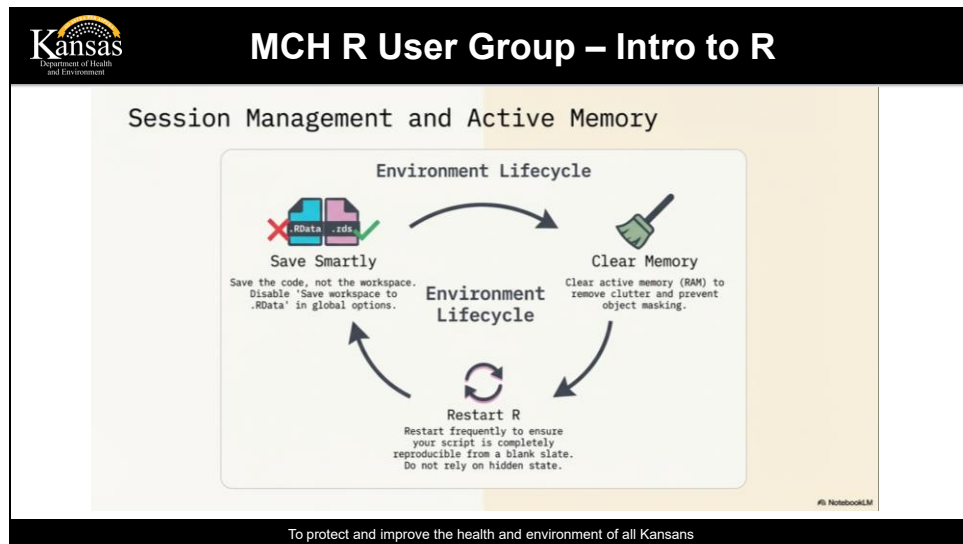


As you saw, with each pane, they have different tabs with different purposes, and we'll only go over a few of the basic and general ones first, but here is a quick summary of what each of the panes does. First the source pane is where you will work with your files. So, think of using each code file, as if you were writing your SAS code file, or writing within a word document editor. The console is the engine that powers and runs R. It processes (or I will sometimes say runs or executes, all meaning the same thing) your code for you to see. You can enter code directly into it or process your script files from above. The environment pane is where your data objects that you create are stored and visible to use. The output pane is where you can see your file directories, for example wherever you are working with your project as well as any visual plots you create and a help section to learn more about different packages and their functions to use.

Slide 5

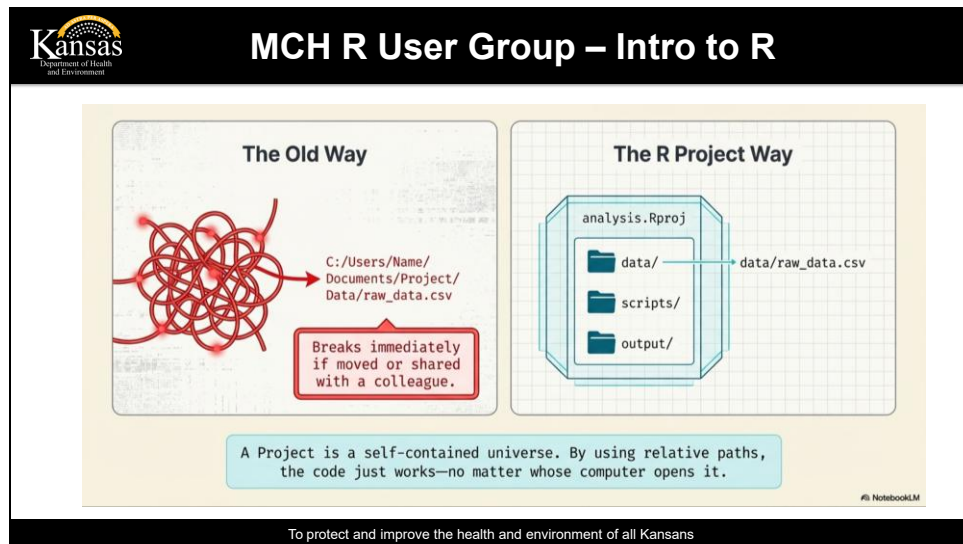


So, on this slide, this is just another illustration of what we talked about, for example in the drafting table pane, the files such as R scripts, markdown and quarto files, you work with are in the source pane, you'll see that blinking cursor symbol in the console on the bottom left, the environment will be where your data frame, functions, or other objects you create go in the top right, and the tabs in the display down at the bottom right will look like what we saw earlier.



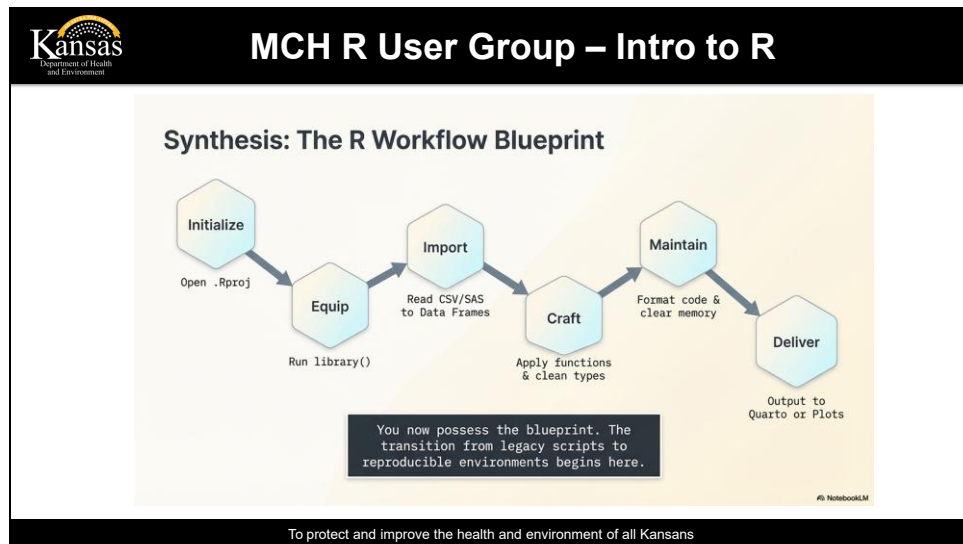
Each time you open R, we call that starting a new session. Since R executes data in memory, sometimes, you may be deep in an analysis and need to clear the memory if you have a lot of things stored or are switching between projects where you don't want the datasets or objects you create to go between projects. So, it works in a similar way when you need to close and restart a program in SAS etc. In R, there is a workspace file that those get saved into called your .RData file, and in most cases, you won't need to save that file. However, there are times when you would want to save the .RData file if you are needing to keep different objects that you create across multiple projects where they may be utilized in both projects, so you don't have to recreate them each time you run the code regardless of project. This just depends on the nature of your project, and how you projects require different data sets.

Slide 7



In SAS, when you read in files, you typically use their paths in the libnames to access datasets, for example on the left. You can do this in R as well, but it's not the most effective or best way to do this. Instead, you can use what are called project files, to keep everything stored and saved in one location. For each analysis you create a project, and within that project you can store your code script files, outputs, and any other data you bring in can all be organized and referenced the same for any user to then be able to take that same project and use as their own, creating a more reproducible and cleaner workflow.

Slide 8



So, bringing some of these concepts into a flow diagram this is what working in a project would look like, so it makes sense from a general analysis framework. For example, for a brand-new project, you would open a new project file, give it a name and save it where you wanted it. Then you would run any library/packages you need to load, which we'll talk about in a second, then bring in your data you're going to use, do your data analysis steps and clean as appropriate, do any types of formatting for your code while clearing old memory if it's needed i.e. if it starts to get bogged down as you build more, and then finally output your charts, reports etc. for your finalized analysis workflow. This helps to make your workflow a good starting foundation for reproducible code and reproducible project analyses.



MCH R User Group – Intro to R

Document Decision Matrix

Scripts (.R)	Markdown (.Rmd)	Quarto (.qmd)
Use Case: Backend data processing, simple functions. Analogy: The rough draft / engine room. 	Use Case: Interleaving text, code, and plot panes. Analogy: The standard analyst notebook. 	Use Case: Modern, multi-format publishing (HTML, PDF, slides). Analogy: The final polished report.  The Modern Standard

© R NotebookLM

To protect and improve the health and environment of all Kansans

So, we mentioned using scripts or code files where you will type out your analysis. In R there are several ways to do that, and the 3 main ways are using a .R script file, a .Rmd R markdown file, and a .qmd Quarto file. Each one will process and execute R code, although slightly differently. Scripts will work the closest to what you may be used to in a SAS program. Code is directly entered into the file, and the entire document can be run. You can enter comment code on this file to communicate notes between code sections. R Markdown and Quarto files work slightly differently. They both work as a traditional word document would, where you can type any text as you see it on the screen. And anywhere you want to write code, you add what is called a code chunk to the page. This section will be the part that processes your code within your overall document, and this is where there is more ability to create full reports and other types of published documents like presentations comes in.



MCH R User Group – Intro to R

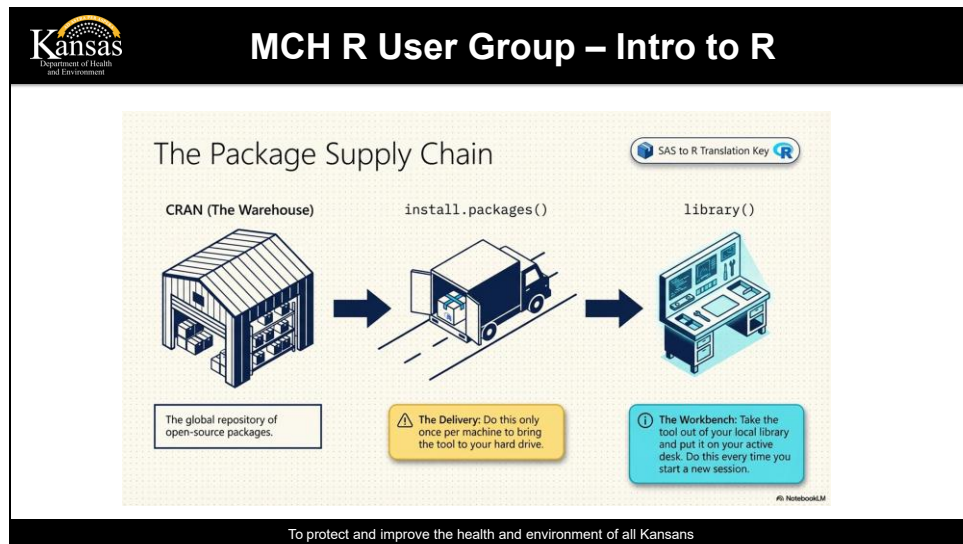
```
{r_setup, include=FALSE}  
knitr::opts_chunk$set(echo = TRUE)
```

Loading Libraries & Data Directories

Here is where you first load the packages.

To protect and improve the health and environment of all Kansans

Now here is an example, of an R chunk alongside regular text within an R Markdown file. You can see the code is separated by the box at the top and it has the green arrow when you want to execute just that section of code in your complete document. In a lot of analyses, as in this example, one of the first things you do is load your package/libraries, and that's what we will talk about next.



So, in R, a package is a set of tools that you will use to load in additional functionality into R. R can do quite a lot of things, but think of packages like web browser extensions, giving R more capabilities in a variety of ways. You only have to install them once, but when you need or want to use them, you then start them in your current session by using the library function.

Slide 12



MCH R User Group – Intro to R

Rainbow Code

```
(function(data){  
  return data.filter(d =>  
    (d.value > 10 &&  
     (d.type === 'A' ||  
      d.type === 'B' && d.status === 'active')));  
})
```

Rainbow brackets prevent missing parenthesis errors and visually untangle complex nesting.

Find / Replace All

```
// remove script variables  
for (variable in window) {  
  if (variable !== 'new_clean_var') {  
    return remove_variable; // old_variable  
  }  
}  
  
// remove variables in document  
for (variable in document) {  
  if (variable !== 'new_clean_var') {  
    return remove_variable; // old_variable  
  }  
}
```

Find: old_variable → Replace: new_clean_var

Instantly transform variable names across a whole script without hunting.

Status Bar & Global Options

Word Count: 1,420 | Hidden Characters | Markdown/Quarto

Word Count: 1,420 | Line: 87, Col: 15

Enable word counts and hidden character views to optimize writing in Markdown and Quarto.

NotebookLM

To protect and improve the health and environment of all Kansans

Now when you start to write your code you have a few settings that are helpful at various times at projects. So, one of them, is what is called rainbow code, and it helps to let you see where each associated ending bracket or parentheses for your functions connect. So, when you start writing code that is longer or more complex over multiple lines, it makes it easier to see where you may have syntax errors to be able fix them easier. Next it has a find/replace feature just like in a word document, so if you have several of the same code bits repeating, you can replace all of them in one go. And last you can see other things like word counts, hidden characters, line numbers etc., all to make it easier to code with.



MCH R User Group – Intro to R

Code Craftsmanship

The Messy Block

```
data mess; set raw;
if x=1 then do;
  y=x*2;
  z=y+3;
  if z>10 then w=z-2; else w=z+2;
end;
else do;
  y=x+3;
  z=y+2; if z<5 then w=z/2;
else w=z*2; end; run;
proc print data=mess; run;
```

The Clean Block

```
# Data Transformation for Final Output
messy_data <-
  raw_data |>
  mutate(
    y = case_when(
      x == 1 ~ x * 2,
      TRUE ~ x * 3
    ),
    z = case_when(
      x == 1 ~ y + 3,
      TRUE ~ y + 2
    ),
    w = case_when(
      z > 10 ~ z - 2,
      z < 5 ~ z / 2,
      TRUE ~ z + 2
    )
  )
```

Commenting: Use # for inline context and to divide code into functional sections.

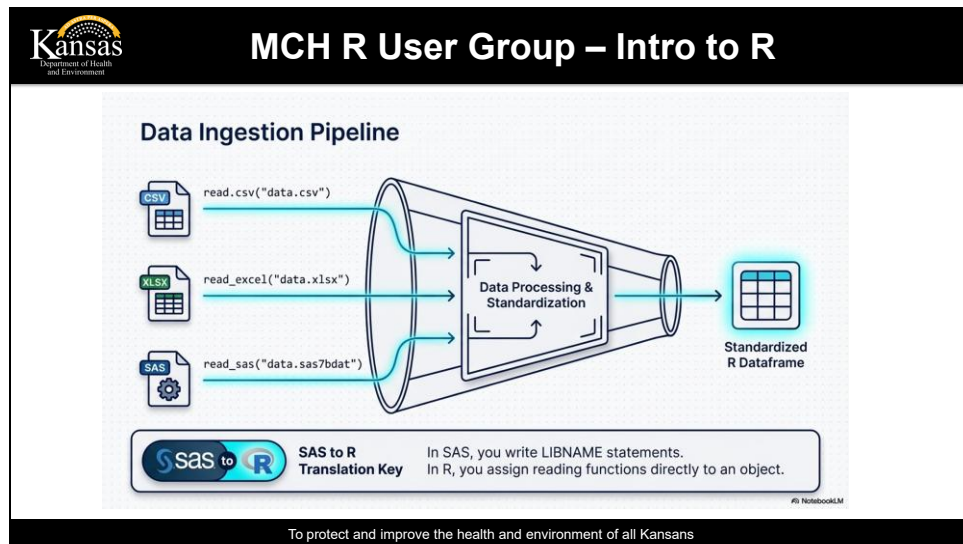
Reflow: Wrapping long text lines for readability.

Re-indent: Automatically aligning nested logic.

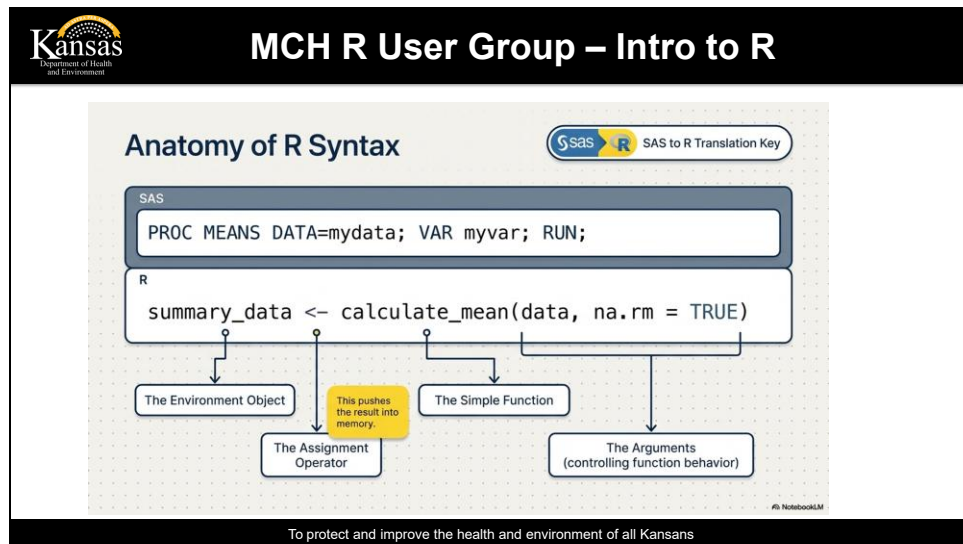
© Notepad++

To protect and improve the health and environment of all Kansans

Now, another important element in general of coding, and not necessarily specific to R, is keeping the code commented and in clean blocked formatting. This allows for easier reading of the code. It also has these additional settings that help you do this with reflow and re-indent, that you can click in the toolbar to help align your code for you so it will straighten it anywhere you didn't type it that way. This allows other users to be able to see and learn from your code if you need to share or are working on a project together.



R will let you import different types of data including csv, excel and even SAS files. This is similar to writing LIBNAME in SAS, but in R, instead you assign each one of your datasets to an object name that you can then do any number of analytic steps on for any other function, which we will talk about next. In this example, it shows the 3 functions that start with read, to bring in each of the 3 datasets shown there.



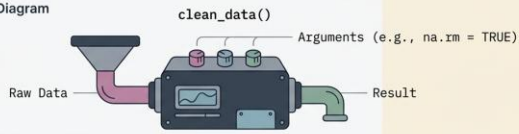
So, the last slide showed you that we can import data, and the `read.csv` or `read_excel` are called functions, and functions are the tools we use to do different steps for the data similar to how you would build out a data step or `proc sql` in SAS. Here, is it how that looks when we examine it: we have the object that we create in our environment, (that's the `summary_data`) and we assign the function (using the arrow); then we have the function (`calculate_mean`, and the arguments to that function, that are like additional settings). We then assign it into the `summary_data` object, so that it will become what we expect, which in this example we are calculating the mean and removing any NAs from the data.



MCH R User Group – Intro to R

Crafting with Functions

The Blueprint Diagram



Code Comparison

Run 1	<code>mean(sales_data, na.rm = FALSE)</code>	NA
Run 2	<code>mean(sales_data, na.rm = TRUE)</code>	42.5

Highlight and run different lines of code sequentially in the Console to instantly compare outputs and tweak function arguments.

© R NotebookLM

To protect and improve the health and environment of all Kansans

So, to visualize functions better, think of it as we are taking the raw data, then doing something to it given the functions arguments, and then it outputs a result. In this example, we've done it two ways, the first, we said FALSE to removing NAs, and then TRUE in the second example, and by doing that, you can see the result ends up looking different. So that is what I mean when I say the arguments can change the way that functions provide the results as in this case where we are taking the mean value of our sample data here.



MCH R User Group – Intro to R

Quick Reference / Cheat Sheet

Top Keyboard Shortcuts

Assignment (<-)	Alt	-
Run Code	Ctrl/Cmd	Enter
Find/Replace All	Ctrl/Cmd	Shift J
Restart Session	Ctrl/Cmd	Shift F10

SAS to R Rosetta Stone Summary

Workspace	→	R Project
LIBNAME	→	read_*() functions
PROC	→	Modular Functions
.sas	→	.R, .Rmd, .qmd

To protect and improve the health and environment of all Kansans

I've gone over a few different things before we explore some of the actual interface, but before that, here are just a few examples of keyboard shortcuts that help make working with the code easier and more productive, but as you can see from the next slide...

Slide 18



MCH R User Group – Intro to R

Keyboard Shortcut Quick Reference

Keyboard	Action
Ctrl+Shift+Tab	Switch to Tab...
Ctrl+F12	Next Tab
Ctrl+F13	Previous Tab
Ctrl+Shift+F11	First Tab
Ctrl+Shift+F12	Last Tab

Panes

Ctrl+S	Move Focus to Source
Ctrl+Z	Move Focus to Console
Ctrl+Y	Move Focus to Help
Ctrl+H	Show History
Ctrl+P	Show Files
Ctrl+M	Show Plots
Ctrl+Z	Show Packages
Ctrl+L	Show Environment
Ctrl+N	Show Viewer
Ctrl+V	Show VIM
Ctrl+R	Show Build
Ctrl+D	Show Connections
Ctrl+O	Show Presentation
Ctrl+F	Show Find in Files
Ctrl+T	Add Source Column
Ctrl+Shift+F	Show All Panels

Files

Ctrl+N	New
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+N	New R Script
Ctrl+Shift+O	Open Recent
Ctrl+Shift+W	Close All
Ctrl+Shift+F	Find in Files...

Source Navigation

Ctrl+B	Back
Ctrl+F	Forward
Ctrl+G	Find Images
Ctrl+H	Use Selection For Find
Ctrl+J	Find
Ctrl+K	Find Next
Ctrl+L	Find Previous
Ctrl+M	Replace and Find
Ctrl+N	Go To File/Function...
Ctrl+O	Go to Line...
Ctrl+P	Jump To...
Ctrl+Q	Jump To Matching
Ctrl+R	Expand To Matching
Ctrl+S	Show Document Outline
Ctrl+T	Add Cursor Above Current Cursor
Ctrl+U	Add Cursor Below Current Cursor
Ctrl+V	Expand Selection
Ctrl+W	Shrink Selection
Ctrl+X	Go to Next Section
Ctrl+Y	Go to Previous Section
Ctrl+Z	Split Into Lines
Ctrl+A	Edit Lines From Start
Ctrl+B	Use Visual Editor
Ctrl+C	Move active cursor up
Ctrl+D	Move active cursor down

Execute

Ctrl+Shift+E	Source Active File
Ctrl+Shift+F	Source with Echo
Ctrl+Shift+P	Preview R
Ctrl+Shift+Q	Preview SQL
Ctrl+Shift+W	Source a File...
Ctrl+Shift+X	Run R Script
Ctrl+Shift+Y	Run Selected Lines()
Ctrl+Shift+Z	Run Lines() without moving cursor
Ctrl+Shift+A	Run All

Source Editor

Ctrl+Shift+I	Insert Chunk
Ctrl+Shift+J	Insert Section...
Ctrl+Shift+K	Extract Function
Ctrl+Shift+L	Extract Variable
Ctrl+Shift+M	Comment/Uncomment Lines
Ctrl+Shift+N	Random Lines
Ctrl+Shift+O	Refactor Comment
Ctrl+Shift+P	Refactor Selection
Ctrl+Shift+Q	Show Diagnostics (Project)
Ctrl+Shift+R	Collapsing Fold
Ctrl+Shift+S	Expanded Fold
Ctrl+Shift+T	Collapsing All Folds
Ctrl+Shift+U	Expanded All Folds
Ctrl+Shift+V	Move Lines Up
Ctrl+Shift+W	Move Lines Down
Ctrl+Shift+X	Delete Line
Ctrl+Shift+Y	Yank Line Up to Cursor
Ctrl+Shift+Z	Yank Line After Cursor
Ctrl+Shift+A	Insert Yanked Text
Ctrl+Shift+B	Insert Assignment Operator
Ctrl+Shift+C	Insert Pipe Operator
Ctrl+Shift+D	Remove in Scope
Ctrl+Shift+E	Insert Rscript Section
Ctrl+Shift+F	Request Copied Code Suggestions

Debug

Ctrl+Shift+B	Toggle Breakpoint
Ctrl+Shift+C	Remove Next Line
Ctrl+Shift+D	Step Into Function
Ctrl+Shift+E	Finish Function/Loop
Ctrl+Shift+F	Continue
Ctrl+Shift+G	Step Debugging

Accessibility

Ctrl+Shift+H	Screen Reader Support
--------------	-----------------------

Source Control

Ctrl+Shift+D	Diff Files
Ctrl+Shift+E	Commit...

Build

Ctrl+Shift+P	Compile PDF
Ctrl+Shift+Q	Preview
Ctrl+Shift+R	Run Document
Ctrl+Shift+S	Render Document
Ctrl+Shift+T	Run Document
Ctrl+Shift+U	Install Package
Ctrl+Shift+V	Load All
Ctrl+Shift+W	Check Package
Ctrl+Shift+X	Test Package
Ctrl+Shift+Y	Document

Console

Ctrl+Shift+Z	Clear Console
Ctrl+Shift+A	Pop Command History

Terminal

Ctrl+Shift+N	New Terminal
Ctrl+Shift+O	Move Focus to Terminal
Ctrl+Shift+P	Previous Terminal
Ctrl+Shift+Q	Next Terminal

Other

Ctrl+Shift+H	Show Function Help
Ctrl+Shift+I	Go To Function / File
Ctrl+Shift+J	Complete Code
Ctrl+Shift+K	Run Script B
Ctrl+Shift+L	Previous Plot
Ctrl+Shift+M	Next Plot
Ctrl+Shift+N	Log focused element
Ctrl+Shift+O	Close All

RStudio: Posit Software PBC

To protect and improve the health and environment of all Kansans

There are lots more, and you can customize and become efficient in them the more you use them. Source: [Posit | The Open-Source Data Science Company](https://posit.co/)



The slide features a black header bar with the Kansas Department of Health and Environment logo on the left and the text "MCH R User Group – Intro to R" on the right. The main content area is white with the text "Now for a Live Demo" centered. A black footer bar at the bottom contains the text "To protect and improve the health and environment of all Kansans".

Kansas
Department of Health
and Environment

MCH R User Group – Intro to R

Now for a Live Demo

To protect and improve the health and environment of all Kansans

Now we can start to demo some of these examples in RStudio to get more familiar with the interface to use R.



MCH R User Group – Intro to R



Jason Geslois

Senior Epidemiologist
Kansas Department of Health and Environment
Jason.geslois@ks.gov

 kdhe.ks.gov  [@KDHEnews](https://www.facebook.com/KDHEnews)  [@kdheks](https://www.instagram.com/kdheks)  [@KDHE](https://twitter.com/KDHE)  [@Kansas Department of Health and Environment](https://www.linkedin.com/company/Kansas%20Department%20of%20Health%20and%20Environment)



Thank You/Questions

